

Azmani Sultana

CSE340

Assignment 03

Id: 22201949

Section : 11

①

0x ABB9609

= 1010 1011 1011 1001 0110 0000 1001

28-bit IEEE 754 format

exponent bit = 6

sign bit = 1

$\therefore$ fraction bit = $28 - 6 - 1 = 21$

sign = -

Biased exponent = 010 101 = 21

Bias = $2^{6-1} - 1 = 31$

$\therefore$ Exponent = $21 - 31 = -10$

Fraction = 1 1011 1001 0110 0000 1001

= 0.110111001011000001001

= 0.8630643615732656

Decimal value = $(-1)^1 \times (1 + 0.86306436157) \times 2^{-10}$
= $-1.86306436157 \times 2^{-10}$

② a) $50.7869 + 79.83 - 29.58$

$$50.7869 = (110010.11001)_2 = 1.1001011001 \times 2^5$$

$$79.83 = (1001111.11010)_2 = 1.00111111010 \times 2^6$$

$$29.58 = (11101.10010)_2 = 1.110110010 \times 2^4$$

$$\begin{aligned} & 1.1001011001 \times 2^5 + 1.00111111010 \times 2^6 - 1.110110010 \times 2^4 \\ &= 0.11001011001 \times 2^6 + 1.00111111010 \times 2^6 - 0.01110110010 \times 2^6 \\ &= (0.11001011001 + 1.00111111010 - 0.01110110010) \times 2^6 \\ &= \cancel{1.100011111} \times 2^6 \\ &= 1.10010 \times 2^6 \end{aligned}$$

b) $64.2486 * 49.1832$

$$= 1000000.00111 * 110001.00101$$

$$= 1.00000000111 \times 2^6 * 1.1000100101 \times 2^5$$

$$= \cancel{1.00000000111 \times 2^6} * \cancel{0.11000100101 \times 2^6}$$

$$= \cancel{(1.00000000111 \times 0.11000100101)} \times 2^6$$

$$= \cancel{0.11000} \times 2^6$$

$$= (1.00000000111 \times 1.1000100101) \times 2^{6+5}$$

$$= 1.100010 \times 2^{11}$$

③

$$28.4810 - (-4.0210)$$

$$= 28.4810 + 4.0210$$

$$= \text{---}$$

$$= 11100.01111 + 100.00000$$

$$= 1.110001111 \times 2^4 + 1.0000000 \times 2^2$$

$$= 1.110001111 \times 2^4 + 0.0010000000 \times 2^4$$

$$= (1.110001111 + 0.0010000000) \times 2^4$$

$$= 10.000001111 \times 2^4$$

$$= 1.000001111 \times 2^5$$

exponent field = 8 bits

$$\text{Bias} = 2^{8-1} - 1 = 127$$

$$\text{Biased exponent} = 127 + 5 = 132$$

$$\text{Range} = 0 \text{ to } 2^8 - 1$$

$$= 0 \text{ to } 255$$

$$= 1 \text{ to } 254 \text{ (reserved 0 and 255)}$$

$$\text{upper range} = 254$$

$$1 \leq 132 \leq 254$$

not overflow and underflow

④a) In IEEE 754 floating point :

A bias is added to the exponent so it can be stored as an unsigned value. This lets the same exponent field represent both positive and negative exponents.

Effect :

positive exponent $\rightarrow$ stored as values above the bias.

Negative exponent $\rightarrow$ stored as values below the bias

This makes comparisons and hardware implementation simpler.

b) Optimized multiplication is faster than traditional long multiplication.

i) It reduces the number of partial products $\rightarrow$ fewer additions.

ii) It handles signed numbers efficiently $\rightarrow$ avoids repeated operations.

iii) It cuts computational complexity $\rightarrow$ less shifting and adding.

⑤ i) What it does :

1. `mul` : Multiplies two 32-bit (or 64-bit) integers and keeps only the lower half of the product (least significant bits).

2. `mulh` : Multiplies two signed integers and keeps the upper half of the product (most significant bits).

3. `mulhu` : Multiplies two unsigned integers and keeps the upper half of the product.

4. `mulhsu` : Multiplies a signed integer (`rs1`) with an unsigned integer (`rs2`) and keeps the upper half of the product.

ii) Syntax :

1. `mul rd, rs1, rs2`

`rd` = destination register

`rs1, rs2` = source registers

2. `mulh rd, rs1, rs2`

3. `mulhu rd, rs1, rs2`

4. `mulhsu rd, rs1, rs2`

iii) Example:

① `mul x5, x6, x7`

$x5 = (x6 * x7)$ lower 32 bits

② `mulh x8, x9, x10`

$x8 =$ upper 32 bits of (signed $x9 * \text{signed } x10$)

③ `mulhu x11, x12, x13`

$x11 =$ upper 32 bits of (unsigned $x12 * \text{unsigned } x13$)

④ `mulhsu x14, x15, x16`

$x14 =$ upper 32 bits of (signed $x15 * \text{unsigned } x16$)

⑥ i) `fle.d` compares 2 floating-point registers

(`$3` and `$5`) and stores the result as an

integer 0 or 1 in an integer register. That's

why the destination is `x5` (an integer register),

not a floating-point register.

ii) `$3` and `$5` both aren't single precision

numbers. The `.d` in `fle.d` stands for double precision.

Correct statement: `$3` and `$5` are double

precision floating-point registers.

(iii) `sltd` sets the destination to 1 if $\$3 \leq \5 , otherwise 0. So after running the instructions, if $x5 = 0$; it means $\$3$ is greater than $\$5$ or one of them is not a number.

⑦

```
seq.d x5, $1, $2
bne x5, x0, jumpEqual
beq x5, x0, jumpNotEqual
```

⑧

```
sgt.d t0, $1, $2
bnez t0, jumpGreater
```

~~seq~~ ~~x0~~, ~~x0~~, ~~x0~~

JumpNotGreater:

```
srli x19, x19, 4
```

JumpGreater